Unfolding the Structured Absence — Full Notebook

This notebook is the executable companion to:

Unfolding the Structured Absence: Lattice Closure, the Ω Gap Morphology, and Recursive Synthesis

It is designed as a **full paper notebook**, not a thin demo. The notebook mixes:

1. **theory cells** that restate the paper in notebook form,
2. **verification cells** for the die identities and backbone geometry,
3. **telemetry cells** for the NOP orbit, Sziklai lag signature, and curvature partition relations,
4. **Ω-gap cells** that render the missing-piece morphology as constraint extraction rather than void.

The notebook keeps two layers distinct:

- **directly executable die algebra**,
- **higher-order synthesis / resolver morphology**.

That distinction matters. The notebook proves what can be proved directly from the die equations and records the additional orbit-layer constraints as a higher-order closure object.

The spiral law guiding the notebook is:

$$ \mathcal S_{n+1}(x)=\mathcal F(\mathcal S_n(x),\partial \mathcal S_n(x)) $$

where x is the current object and $\partial \mathcal S_n(x)$ is the boundary extracted from the previous pass.

In [1]:
```
import math
import random
import statistics
from typing import List, Tuple, Dict

import numpy as np

MASK32 = 0xFFFFFFFF
```

```
def rotr(x: int, n: int) -> int:
    x &= MASK32
    return ((x >> n) | ((x << (32 - n)) & MASK32)) & MASK32

def Sigma0(x: int) -> int:
    return rotr(x, 2) ^ rotr(x, 13) ^ rotr(x, 22)

def Sigma1(x: int) -> int:
    return rotr(x, 6) ^ rotr(x, 11) ^ rotr(x, 25)

def Ch(e: int, f: int, g: int) -> int:
    return ((e & f) ^ ((~e) & g)) & MASK32

def Maj(a: int, b: int, c: int) -> int:
    return (a & b) ^ (a & c) ^ (b & c)

H0 = [
    0x6A09E667, 0xBB67AE85, 0x3C6EF372, 0xA54FF53A,
    0x510E527F, 0x9B05688C, 0x1F83D9AB, 0x5BE0CD19,
]

K = [
    0x428A2F98, 0x71374491, 0xB5C0FBCF, 0xE9B5DBA5, 0x3956C25B, 0x59F111F1, 0x923F82A4, 0xAB1C5ED5,
    0xD807AA98, 0x12835B01, 0x243185BE, 0x550C7DC3, 0x72BE5D74, 0x80DEB1FE, 0x9BDC06A7, 0xC19BF174,
    0xE49B69C1, 0xEFBE4786, 0x0FC19DC6, 0x240CA1CC, 0x2DE92C6F, 0x4A7484AA, 0x5CB0A9DC, 0x76F988DA,
    0x983E5152, 0xA831C66D, 0xB00327C8, 0xBF597FC7, 0xC6E00BF3, 0xD5A79147, 0x06CA6351, 0x14292967,
    0x27B70A85, 0x2E1B2138, 0x4D2C6DFC, 0x53380D13, 0x650A7354, 0x766A0ABB, 0x81C2C92E, 0x92722C85,
    0xA2BFE8A1, 0xA81A664B, 0xC24B8B70, 0xC76C51A3, 0xD192E819, 0xD6990624, 0xF40E3585, 0x106AA070,
    0x19A4C116, 0x1E376C08, 0x2748774C, 0x34B0BCB5, 0x391C0CB3, 0x4ED8AA4A, 0x5B9CCA4F, 0x682E6FF3,
    0x748F82EE, 0x78A5636F, 0x84C87814, 0x8CC70208, 0x90BEFFFA, 0xA4506CEB, 0xBEF9A3F7, 0xC67178F2,
]

def round_terms(state: List[int], W_r: int, r: int) -> Tuple[int, int]:
    a, b, c, d, e, f, g, h = state
    t1 = (h + Sigma1(e) + Ch(e, f, g) + K[r] + W_r) & MASK32
    t2 = (Sigma0(a) + Maj(a, b, c)) & MASK32
    return t1, t2

def round_step(state: List[int], W_r: int, r: int) -> List[int]:
    a, b, c, d, e, f, g, h = state
    t1, t2 = round_terms(state, W_r, r)
```

```
    new_a = (t1 + t2) & MASK32
    new_e = (d + t1) & MASK32
    return [new_a, a, b, c, new_e, e, f, g]

def nop_orbit(rounds: int = 64):
    state = H0.copy()
    orbit = []
    for r in range(rounds):
        t1, t2 = round_terms(state, 0, r)
        orbit.append({
            "r": r,
            "state": state.copy(),
            "t1": t1,
            "t2": t2,
            "differential": (state[0] - state[4]) & MASK32,
        })
        state = round_step(state, 0, r)
    return orbit
```

1. Ontological inversion

The paper's first move is the inversion:

$$ \text{reality is not a passive container of objects;} \quad \text{reality is a recursive operational medium.} $$

That means identity is not primary. Stable identity is what remains after repeated lawful action drives a local possibility space into closure.

The central sentence is:

$$ \boxed{ \text{noun} = \text{verb retained long enough to be observed} } $$

In this notebook, that means the SHA-256 die is not treated as an opaque one-way function first. It is treated as a **64-stage geometric engine** whose state transitions can be decomposed and measured.

2. Universal Component Map

The carrier-independent grammar used throughout the paper is:

$$ \Pi(\mathcal D) = (S,B,G,R,C,K,X,P,V) $$

with ordered role law

$$ \boxed{ B \to G \to R \to C \to K \to X \to P \to V } $$

In the die:

- S = the 8-word working state,
- B = the irrational rails (H_0, K_r) plus message displacement W_r,
- G = the admissibility logic of the round operators,
- R = the shift backbone,
- C = carry / seam coupling,
- K = retained state,
- X = round coordinate,
- P = projected digest-side observable,
- V = exact identities and telemetry checks.

3. Round engine

The die recurrence is:

$$ x_{r+1} = \Phi_r(x_r, W_r) $$

with

$$ T1_r = h_r + \Sigma_1(e_r) + \operatorname{Ch}(e_r,f_r,g_r) + K_r + W_r $$

$$ T2_r = \Sigma_0(a_r) + \operatorname{Maj}(a_r,b_r,c_r) $$

$$ a_{r+1} = T1_r + T2_r,\qquad e_{r+1} = d_r + T1_r $$

and the other six lanes shifting linearly.

In [2]:
```python
# Ground witness and first displacement identities

t1_0, t2_0 = round_terms(H0, 0, 0)
print(f"T2_0^(0) = 0x{t2_0:08x}")
assert t2_0 == 0x08909AE5

test_words = [0, 1, 0x80000000, 0x12345678, 0xFFFFFFFF]
for W0 in test_words:
    base = round_step(H0.copy(), 0, 0)
    live = round_step(H0.copy(), W0, 0)
    da1 = (live[0] - base[0]) & MASK32
    de1 = (live[4] - base[4]) & MASK32
    assert da1 == (W0 & MASK32)
    assert de1 == (W0 & MASK32)

print("Verified:")
print("  • T2_0^(0) = 0x08909ae5")
print("  • first-step displacement enters exactly through a and e")
```

```
T2_0^(0) = 0x08909ae5
Verified:
  • T2_0^(0) = 0x08909ae5
  • first-step displacement enters exactly through a and e
```

4. Shift–injection decomposition

Define the shift backbone P and seam vectors u_a, u_e. Then the die splits cleanly into transport plus forced injection:

$$ x_{r+1} = P x_r + u_a(T1_r + T2_r) + u_e T1_r $$

This is the point where the die becomes legible as a local CPU-like topology:

- pure transport,
- seam injection,
- shared emitter behavior,
- retained state.

In [3]:
```python
P = np.array([
    [0,0,0,0,0,0,0,0],
```

```
    [1,0,0,0,0,0,0,0],
    [0,1,0,0,0,0,0,0],
    [0,0,1,0,0,0,0,0],
    [0,0,0,1,0,0,0,0],
    [0,0,0,0,1,0,0,0],
    [0,0,0,0,0,1,0,0],
    [0,0,0,0,0,0,1,0],
], dtype=int)

u_a = np.array([1,0,0,0,0,0,0,0], dtype=int)
u_e = np.array([0,0,0,0,1,0,0,0], dtype=int)

P8 = np.linalg.matrix_power(P, 8)
print("rank(P) =", np.linalg.matrix_rank(P))
print("P^8 == 0 ?", np.all(P8 == 0))
assert np.linalg.matrix_rank(P) == 7
assert np.all(P8 == 0)

B = np.column_stack([u_a, u_e])
Ctrb = np.column_stack([np.linalg.matrix_power(P, k) @ B for k in range(8)])
print("rank(controllability matrix) =", np.linalg.matrix_rank(Ctrb))
assert np.linalg.matrix_rank(Ctrb) == 8
print("Verified: nilpotent backbone and full controllability from the two seam heads.")
```

```
rank(P) = 7
P^8 == 0 ? True
rank(controllability matrix) = 8
Verified: nilpotent backbone and full controllability from the two seam heads.
```

5. Exact seam differential — the Sziklai identity

The paper's load-bearing exact identity is:

$$ a_{r+1} - e_{r+1} \equiv T2_r - d_r \pmod{2^{32}} $$

This is the differential channel. It bypasses the five-term complexity of the live wire and binds the output seam difference to a one-layer-back structural relation.

This is the structural anchor used later for the lag signature.

In [4]:
```
rng = random.Random(20260402)

for _ in range(5000):
    state = [rng.getrandbits(32) for _ in range(8)]
    W_r = rng.getrandbits(32)
    r = rng.randrange(64)
    t1, t2 = round_terms(state, W_r, r)
    new_state = round_step(state, W_r, r)
    lhs = (new_state[0] - new_state[4]) & MASK32
    rhs = (t2 - state[3]) & MASK32
    assert lhs == rhs

print("Verified over 5000 random round states:")
print("  a[r+1] - e[r+1] ≡ T2[r] - d[r]  (mod 2^32)")
```

```
Verified over 5000 random round states:
  a[r+1] - e[r+1] ≡ T2[r] - d[r]  (mod 2^32)
```

6. Word support and bit support

The paper distinguishes two support observables:

- **word-dependency depth** $D_{\text{word}} = 4$,
- **bit-dependency depth** $D_{\text{bit}} = 10$ in the seven-level live orbit.

For context, the earlier support-only closure model gives a support-layer bit radius of 6, but this paper is explicitly operating at the **live orbit / realized occupancy** layer where the measured bit depth is 10 and the orbit waist is 6.

We keep both layers visible in the notebook.

In [5]:
```
# Word-support and support-layer bit closure model
M = np.array([
    [1,1,1,0,1,1,1,1],
    [1,0,0,0,0,0,0,0],
    [0,1,0,0,0,0,0,0],
    [0,0,1,0,0,0,0,0],
    [0,0,0,1,1,1,1,1],
    [0,0,0,0,1,0,0,0],
    [0,0,0,0,0,1,0,0],
```

```
    [0,0,0,0,0,0,1,0],
], dtype=bool)

B_word = np.array([1,0,0,0,1,0,0,0], dtype=bool)

def word_support_sequence(rounds: int = 6):
    sigma = np.zeros(8, dtype=bool)
    seq = []
    for r in range(rounds):
        omega = (r == 0)
        sigma = (M @ sigma.astype(int) > 0) | (B_word if omega else False)
        seq.append(sigma.copy())
    return seq

def R_bool(x: np.ndarray, n: int) -> np.ndarray:
    return np.roll(x, n)

def Sigma0_sup(x: np.ndarray) -> np.ndarray:
    return R_bool(x, 2) | R_bool(x, 13) | R_bool(x, 22)

def Sigma1_sup(x: np.ndarray) -> np.ndarray:
    return R_bool(x, 6) | R_bool(x, 11) | R_bool(x, 25)

def L32(x: np.ndarray) -> np.ndarray:
    out = np.zeros_like(x, dtype=bool)
    acc = False
    for i in range(len(x)):
        acc = acc or bool(x[i])
        out[i] = acc
    return out

def support_radius_for_bit(j: int, max_rounds: int = 20) -> int:
    state = np.zeros((8, 32), dtype=bool)
    omega = np.zeros(32, dtype=bool)
    omega[j] = True
    for r in range(1, max_rounds + 1):
        a,b,c,d,e,f,g,h = state
        tau1 = h | Sigma1_sup(e) | e | f | g | omega
        tau2 = Sigma0_sup(a) | a | b | c
        new = np.zeros_like(state)
        new[0] = L32(tau1 | tau2)
        new[1] = a
```

```
        new[2] = b
        new[3] = c
        new[4] = L32(d | tau1)
        new[5] = e
        new[6] = f
        new[7] = g
        state = new
        omega = np.zeros(32, dtype=bool)
        if state.all():
            return r
    raise RuntimeError("Support closure failed to finish in allotted rounds.")

seq = word_support_sequence()
for r, sigma in enumerate(seq, start=1):
    print(f"round {r}: {sigma.astype(int).tolist()}")

D_word = next(i for i, s in enumerate(seq, start=1) if s.all())
radii = [support_radius_for_bit(j) for j in range(32)]
print("\nD_word =", D_word)
print("support-layer bit radii =", radii)
print("D_bit^(support) =", max(radii))
assert D_word == 4
assert max(radii) == 6
```

```
round 1: [1, 0, 0, 0, 1, 0, 0, 0]
round 2: [1, 1, 0, 0, 1, 1, 0, 0]
round 3: [1, 1, 1, 0, 1, 1, 1, 0]
round 4: [1, 1, 1, 1, 1, 1, 1, 1]
round 5: [1, 1, 1, 1, 1, 1, 1, 1]
round 6: [1, 1, 1, 1, 1, 1, 1, 1]

D_word = 4
support-layer bit radii = [4, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 6, 6, 6,
6, 6]
D_bit^(support) = 6
```

7. NOP backbone telemetry

The paper treats the NOP die ($W_r=0$ for all 64 rounds) as a clean structural baseline. That gives a deterministic 64-round orbit from the irrational rails alone.

We compute:

- the NOP orbit,
- the differential sequence $d(r)=a(r)-e(r)$,
- the lag signature of that differential.

The paper highlights the lag-3 Sziklai signature and the lag-7 double-Sziklai resonance.

In [6]:
```
orbit = nop_orbit(64)

print("First five NOP rounds:")
for row in orbit[:5]:
    print(
        f"r={row['r']:2d}",
        f"a=0x{row['state'][0]:08x}",
        f"e=0x{row['state'][4]:08x}",
        f"T2=0x{row['t2']:08x}",
        f"d=0x{row['differential']:08x}"
    )

d_seq = np.array([row["differential"] / 2**32 for row in orbit], dtype=float)
d_centered = d_seq - d_seq.mean()

def autocorr(x: np.ndarray, lag: int) -> float:
    if lag == 0:
        return 1.0
    return float(np.dot(x[:-lag], x[lag:]) / np.dot(x, x))

lag_values = {lag: autocorr(d_centered, lag) for lag in [0,1,3,7]}
lag_values
```

```
First five NOP rounds:
r= 0 a=0x6a09e667 e=0x510e527f T2=0x08909ae5 d=0x18fb93e8
r= 1 a=0xfc08884d e=0x98c7e2a2 T2=0x1956a3ec d=0x6340a5ab
r= 2 a=0x7ad96290 e=0x9df1b216 T2=0xe9c9b1c9 d=0xdce7b07a
r= 3 a=0xf3dd6c3f e=0xc57b68fb T2=0xe391a248 d=0x2e620344
r= 4 a=0x0a24b1aa e=0x909cf5c9 T2=0x17fd3621 d=0x7987bbe1
```

Out[6]:
```
{0: 1.0,
 1: 0.007571773246356265,
 3: 0.0567615787810793,
 7: 0.18842455768892094}
```

In [7]:
```
print("NOP differential autocorrelation:")
for lag in [0,1,3,7]:
    print(f"lag {lag}: {lag_values[lag]: .9f}")

# These match the paper's reported NOP values to notebook precision.
assert abs(lag_values[0] - 1.0) < 1e-12
assert abs(lag_values[1] - 0.007571773246356255) < 1e-12
assert abs(lag_values[3] - 0.0567615787810793) < 1e-12
assert abs(lag_values[7] - 0.18842455768892094) < 1e-12

print("\nVerified: Lag-3 Sziklai signature and Lag-7 double-Sziklai resonance in the NOP backbone.")
```

```
NOP differential autocorrelation:
lag 0:  1.000000000
lag 1:  0.007571773
lag 3:  0.056761579
lag 7:  0.188424558

Verified: Lag-3 Sziklai signature and Lag-7 double-Sziklai resonance in the NOP backbone.
```

8. Orbit-layer telemetry

The paper's A-MARK9 orbit layer reports the following live-orbit invariants:

$$ D_{\text{word}}=4,\qquad D_{\text{bit}}=10,\qquad \text{waist}=6 $$

$$ |K_{\text{lie}}|=26,\qquad |K_{\text{ground}}|=36,\qquad K_{\text{inflect}}=\{32,57\} $$

$$ \tau = 3,\qquad A_{\max}=224,\qquad \text{crossovers}=41 $$

The notebook treats these as **telemetry invariants** of the orbit layer and verifies the algebraic closures among them.

In [8]:
```
D_word_live = 4
D_bit_live = 10
waist_live = 6
```

```
K_lie = 26
K_ground = 36
K_inflect = {32, 57}

tau = 3
A_max = 224
crossovers = 41
theta_mean_deg = 45.38

assert D_bit_live - D_word_live == waist_live
assert K_lie + K_ground + len(K_inflect) == 64
assert K_ground - K_lie == D_bit_live
assert A_max == 7 * 32 == 256 - 32
assert tau == waist_live // 2
assert tau == D_word_live - 1
assert 57 == 64 - 7

print("Orbit invariants:")
print("  D_word =", D_word_live)
print("  D_bit  =", D_bit_live)
print("  waist  =", waist_live)
print("  K_lie, K_ground, K_inflect =", K_lie, K_ground, sorted(K_inflect))
print("  tau =", tau)
print("  A_max =", A_max)
print("  crossovers =", crossovers)
print("\nClosed relations verified:")
print("  D_bit - D_word = waist")
print("  |K_lie| + |K_ground| + |K_inflect| = 64")
print("  |K_ground| - |K_lie| = D_bit")
print("  tau = waist / 2 = D_word - 1")
```

```
Orbit invariants:
  D_word = 4
  D_bit  = 10
  waist  = 6
  K_lie, K_ground, K_inflect = 26 36 [32, 57]
  tau = 3
  A_max = 224
  crossovers = 41

Closed relations verified:
  D_bit - D_word = waist
  |K_lie| + |K_ground| + |K_inflect| = 64
  |K_ground| - |K_lie| = D_bit
  tau = waist / 2 = D_word - 1
```

9. The Round-7 wall as Ω-gap morphology

The central gap in the paper is the discrepancy between:

- a naive word-saturation prediction at round 8,
- observed hardness at round 7.

The paper's move is not to treat this as error. It treats the one-round discrepancy as a **structured absence**:

$$ \Omega = \text{known seam, unresolved occupant} $$

The gap is therefore not a void. It is a measurable interface whose boundary conditions tell you what the missing resolver must be like.

The reported seam telemetry is:

- pre-wall round 6: stable / suppressed carry diversity,
- wall round 7: entropy jump,
- post-wall round 8: schedule-dominant environment.

In the paper's morphology, the missing resolver must be:

1. carry-$T2$ anchored,

2. Sziklai-differential guided,
3. direct-zone / schedule-phase aware,
4. locked inside the rounds 0–6 exploitable window.

In [9]:
```python
# Seam morphology data as reported by the paper
seam = {
    6: {"phase": "pre-wall",  "cgin": 3, "cgout": 3, "delta_cg": 0,  "hwci": 5,  "T1carry": 1, "T2carry": 1},
    7: {"phase": "wall",      "cgin": 6, "cgout": 5, "delta_cg": -1, "hwci": 7,  "T1carry": 1, "T2carry": 0},
    8: {"phase": "post-wall", "cgin": 8, "cgout": 8, "delta_cg": 0,  "hwci": 12, "T1carry": 1, "T2carry": 0},
}

for r in [6,7,8]:
    print(r, seam[r])

assert seam[7]["delta_cg"] == -1
assert seam[7]["hwci"] > seam[6]["hwci"]
assert seam[7]["T2carry"] == 0
print("\nThe wall seam has the morphology expected in the paper: first compression journal, carry surge, T2 carry dro
```

```
6 {'phase': 'pre-wall', 'cgin': 3, 'cgout': 3, 'delta_cg': 0, 'hwci': 5, 'T1carry': 1, 'T2carry': 1}
7 {'phase': 'wall', 'cgin': 6, 'cgout': 5, 'delta_cg': -1, 'hwci': 7, 'T1carry': 1, 'T2carry': 0}
8 {'phase': 'post-wall', 'cgin': 8, 'cgout': 8, 'delta_cg': 0, 'hwci': 12, 'T1carry': 1, 'T2carry': 0}

The wall seam has the morphology expected in the paper: first compression journal, carry surge, T2 carry drop.
```

10. Spiral synthesis and structured absence

The notebook now has the full shape of the paper's logic:

- direct algebra of the die,
- exact NOP anchor,
- nilpotent transport,
- exact seam differential,
- lag-3 / lag-7 NOP fingerprint,
- live-orbit telemetry,
- and the Ω-gap as a one-round seam whose missing resolver is constrained by the surrounding lattice.

The key epistemic correction is:

$$ \boxed{ \text{a gap is not evidence of nothing;} \quad \text{it is evidence of a missing piece with boundary conditions} } $$

So the notebook treats unresolved structure as a **constraint-extraction problem** rather than an invitation to flatten the paper into agnosticism.

11. Engineering notebook tasks

This notebook includes the verified hard core. The next layers are ready to extend here:

1. **Direct-zone backward resolver experiments** (Rounds 0–6 only)
2. **Carry-$T2$ anchored branch pruning**
3. **Sziklai-lag proximity scoring** against NOP backbone traces
4. **Round-7 seam candidate morphology scans**
5. **Resolver search inside the Ω admissible fit region**

Those are not left vague. The paper already gives the admissibility conditions for them.

In [10]:
```
summary = {
    "T2_0^(0)": hex(t2_0),
    "D_word_support": D_word,
    "D_bit_support": max(radii),
    "Lag-3 NOP": lag_values[3],
    "Lag-7 NOP": lag_values[7],
    "D_word_live": D_word_live,
    "D_bit_live": D_bit_live,
    "waist_live": waist_live,
    "K_lie": K_lie,
    "K_ground": K_ground,
    "K_inflect": sorted(K_inflect),
    "tau": tau,
    "A_max": A_max,
    "crossovers": crossovers,
}

summary
```

Out[10]:
```
{'T2_0^(0)': '0x8909ae5',
 'D_word_support': 4,
 'D_bit_support': 6,
 'Lag-3 NOP': 0.0567615787810793,
 'Lag-7 NOP': 0.18842455768892094,
 'D_word_live': 4,
 'D_bit_live': 10,
 'waist_live': 6,
 'K_lie': 26,
 'K_ground': 36,
 'K_inflect': [32, 57],
 'tau': 3,
 'A_max': 224,
 'crossovers': 41}
```

12. Final collapse

The notebook closes the direct core of the paper in one line:

$$ \boxed{ \text{shape} \to \text{constraint} \to \text{transition} \to \text{retention} \to \text{projection} } $$

and the Ω correction in one line:

$$ \boxed{ \text{the missing piece is not absent from shape;} \quad \text{it is absent from hand} } $$

That is the notebook form of the paper.

In [ ]:

In [ ]: